

AI BlogNest API

Project Demonstration Report

Project Demonstration Phase

Project Name	AI BlogNest API
Team ID	SWTID-2026-6999
Date	26 September 2026

1. Demonstration Objective

The objective of the project demonstration is to demonstrate the working functionality of the AI BlogNest API and show how the different backend modules operate together.

The demonstration covers authentication, role-based access control, blog management, categories, comments, Google Gemini AI integration, keyword search, semantic search, and MongoDB Atlas Vector Search.

The BlogNest project specification identifies AI blog generation, summarization, blog management, role-based access, and API execution as key parts of the system.

2. Demonstration Environment

The demonstration is performed using:

- Visual Studio Code
- Node.js
- Express.js
- MongoDB Atlas
- Google Gemini AI
- Postman

The backend is accessed through:

http://localhost:8000

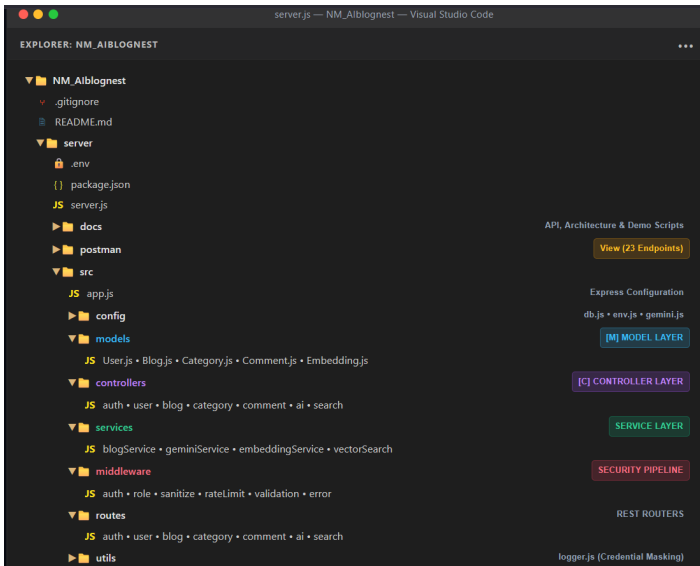


Figure 1. Visual Studio Code project structure showing models, controllers, services, middleware, and routes.

3. Demonstration Flow

The project is demonstrated in the following sequence, shown below:



Figure 2. Demonstration flow from backend startup and health check through authentication, blog management, AI functions, and search.

4. Backend Startup

The backend server is started and the API is made available for testing.

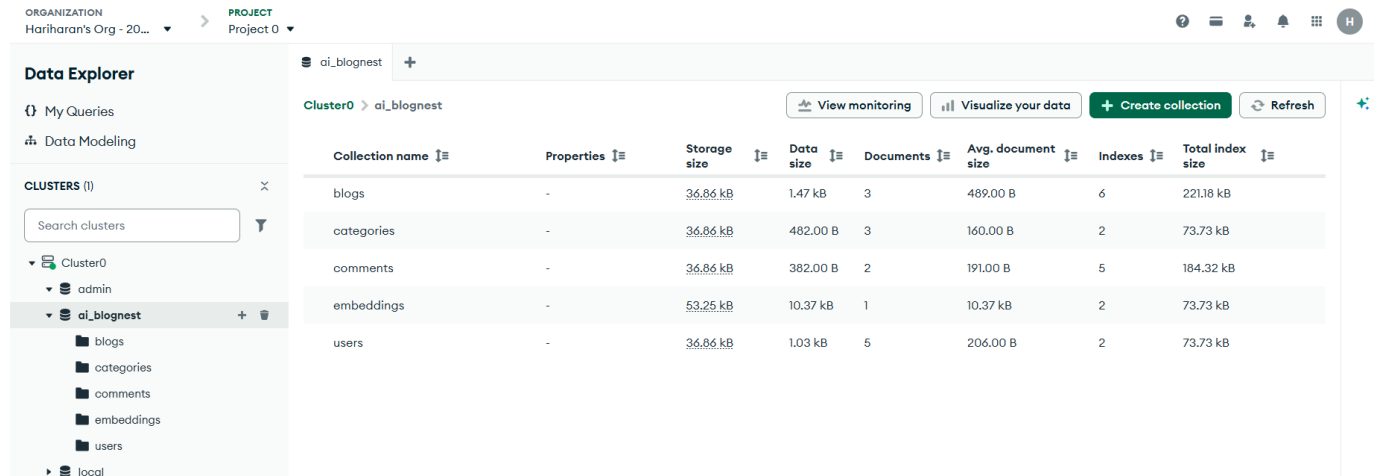
The health-check endpoint is used to verify that the application is running successfully.

GET /api/health

Expected result:

```
{
  "success": true,
  "message": "AI BlogNest API is running"
}
```

The database view below shows the MongoDB Atlas collections used by the backend.



The screenshot shows the MongoDB Atlas Data Explorer interface. On the left, the 'Data Explorer' sidebar shows the hierarchy: Organization (Hariharan's Org - 20...), Project (Project 0), Cluster (Cluster0), and Database (ai_blognest). The main panel displays a table of collections for the 'ai_blognest' database. The table has columns for Collection name, Properties, Storage size, Data size, Documents, Avg. document size, Indexes, and Total index size. The collections listed are blogs, categories, comments, embeddings, and users.

Collection name	Properties	Storage size	Data size	Documents	Avg. document size	Indexes	Total index size
blogs	-	36.86 kB	1.47 kB	3	489.00 B	6	221.18 kB
categories	-	36.86 kB	482.00 B	3	160.00 B	2	73.73 kB
comments	-	36.86 kB	382.00 B	2	191.00 B	5	184.32 kB
embeddings	-	53.25 kB	10.37 kB	1	10.37 kB	2	73.73 kB
users	-	36.86 kB	1.03 kB	5	206.00 B	2	73.73 kB

Figure 3. MongoDB Atlas collections used by the running backend: blogs, categories, comments, embeddings, and users.

5. Authentication Demonstration

The registration API is demonstrated first.

POST /api/auth/register

A user is created using valid registration details.

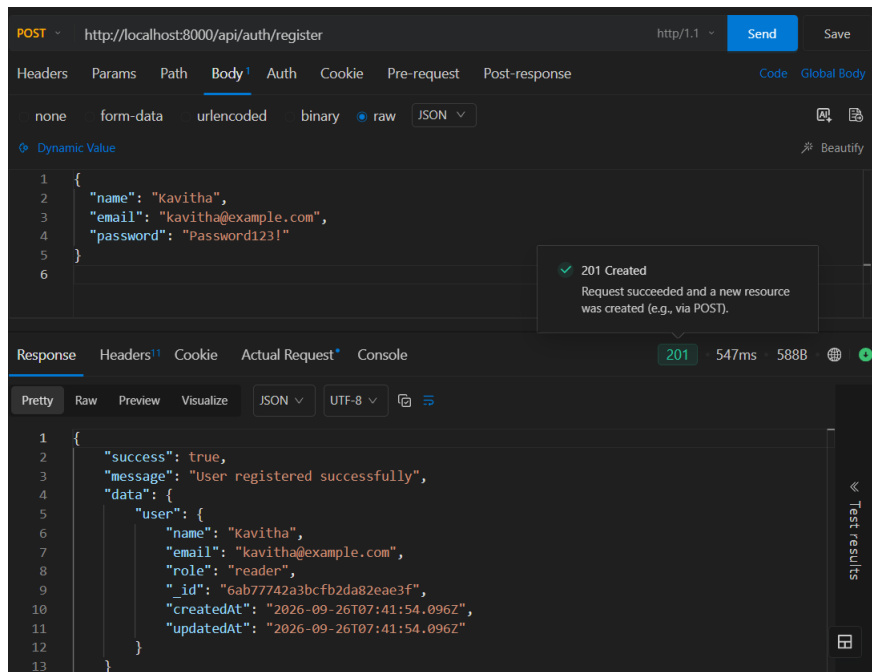


Figure 4. Registration request returning 201 Created and a successfully registered user.

Next, the login API is demonstrated:

POST /api/auth/login

A JWT token is returned after successful authentication. The token is then used to access protected APIs.

The BlogNest specification includes registration, login, and JWT-based authentication.

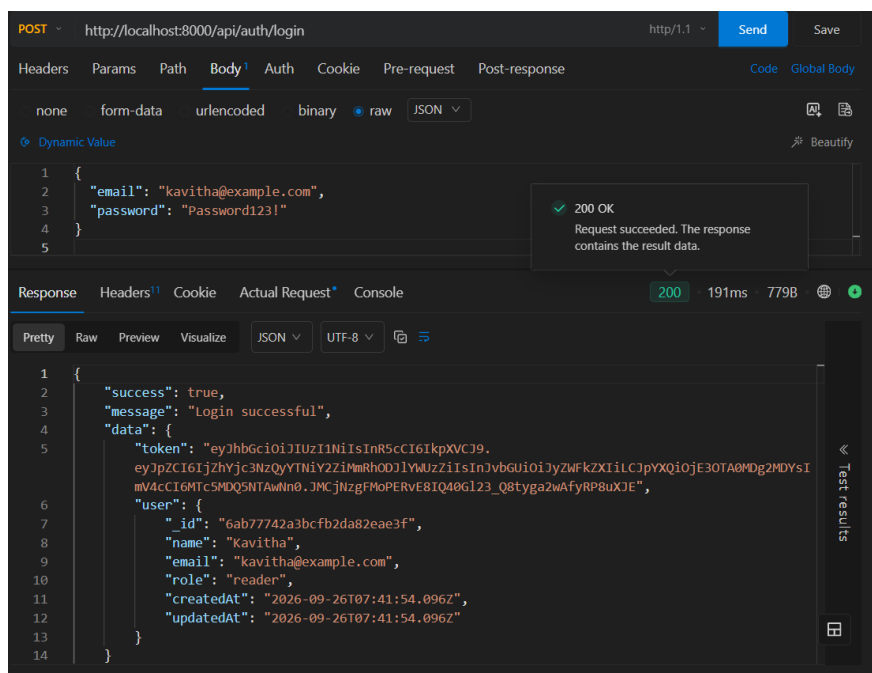


Figure 5. Login request returning 200 OK, a JWT token, and the authenticated user details.

6. Role-Based Access Control Demonstration

The four supported roles are:

- Admin
- Editor
- Author
- Reader

The demonstration shows that different users have different permissions. For example:

- An Author can create a blog.
- A Reader cannot create a blog.
- An Admin can manage users and system resources.
- An Editor can review and manage content according to permissions.

The implementation includes role middleware and ownership protection.

```
server > src > middleware > JS roleMiddleware.js > ...
1  /**
2   * Middleware to enforce Role-Based Access Control (RBAC)
3   * @param {...string} roles - List of allowed roles (e.g. 'admin', 'editor', 'author', 'reader')
4   */
5  const requireRole = (...roles) => {
6    return (req, res, next) => {
7      if (!req.user) {
8        return res.status(401).json({
9          success: false,
10         message: 'Authentication required before checking role permissions'
11       });
12     }
13
14     if (!roles.includes(req.user.role)) {
15       return res.status(403).json({
16         success: false,
17         message: `Forbidden: Access restricted to [${roles.join(', ')}]. Your role is '${req.user.role}'`
18       });
19     }
20
21     next();
22   };
23 };
24
25 module.exports = {
26   requireRole
27 };
28
```

Figure 6. Role middleware checking authentication and permitted roles before allowing protected operations.

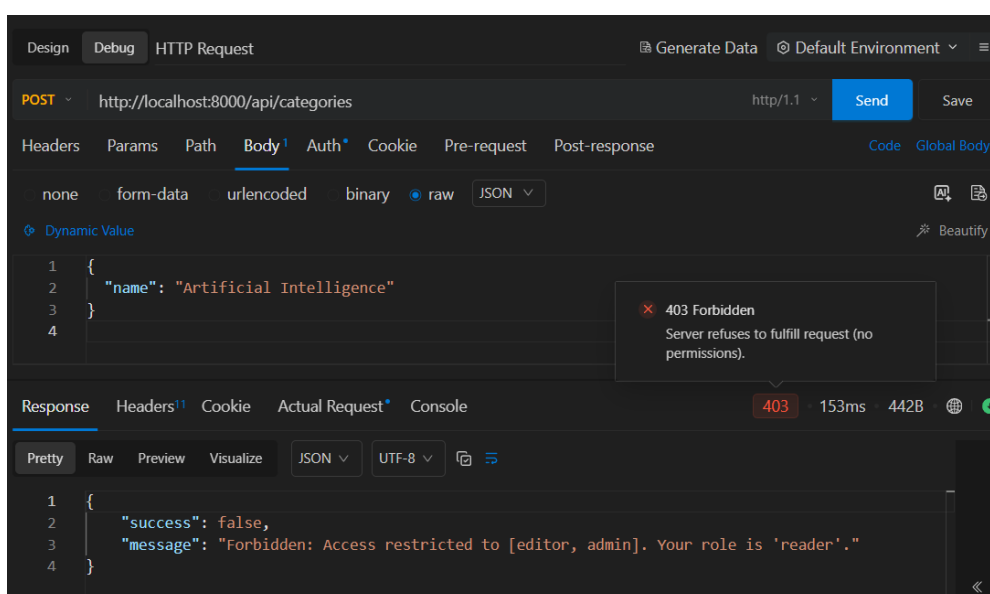


Figure 7. A Reader attempting category creation receives 403 Forbidden because access is restricted to Editor and Admin roles.

7. Blog Management Demonstration

The blog management APIs are demonstrated using Postman.

```
POST    /api/blogs
GET     /api/blogs
GET     /api/blogs/:id
PUT     /api/blogs/:id
DELETE  /api/blogs/:id
```

The demonstration shows blog creation, retrieval, modification, and deletion according to user permissions.

The BlogNest specification defines blog lifecycle management and CRUD operations.

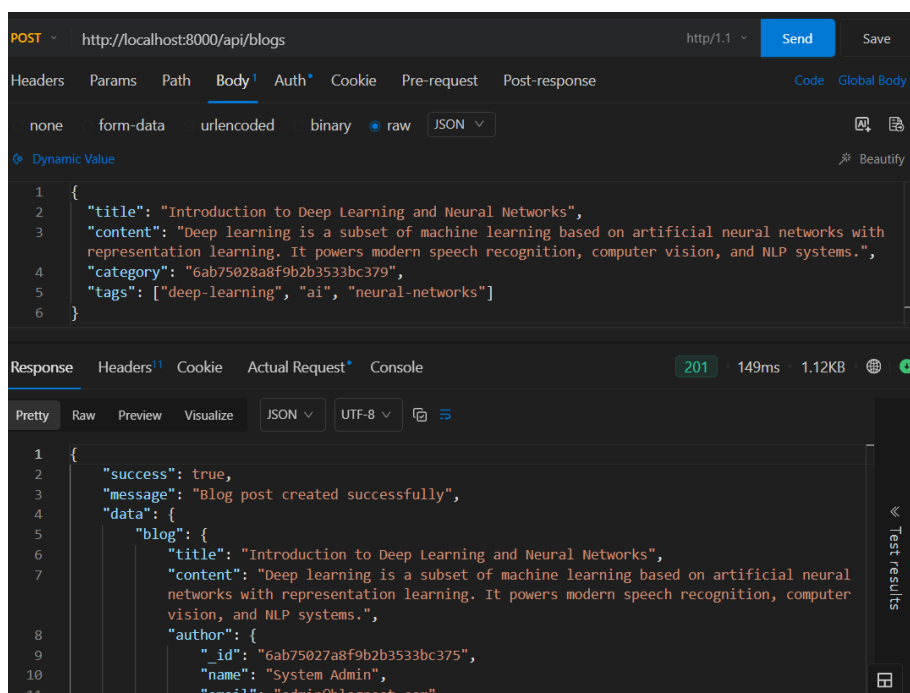


Figure 8. Blog creation request returning 201 Created with the saved blog data.

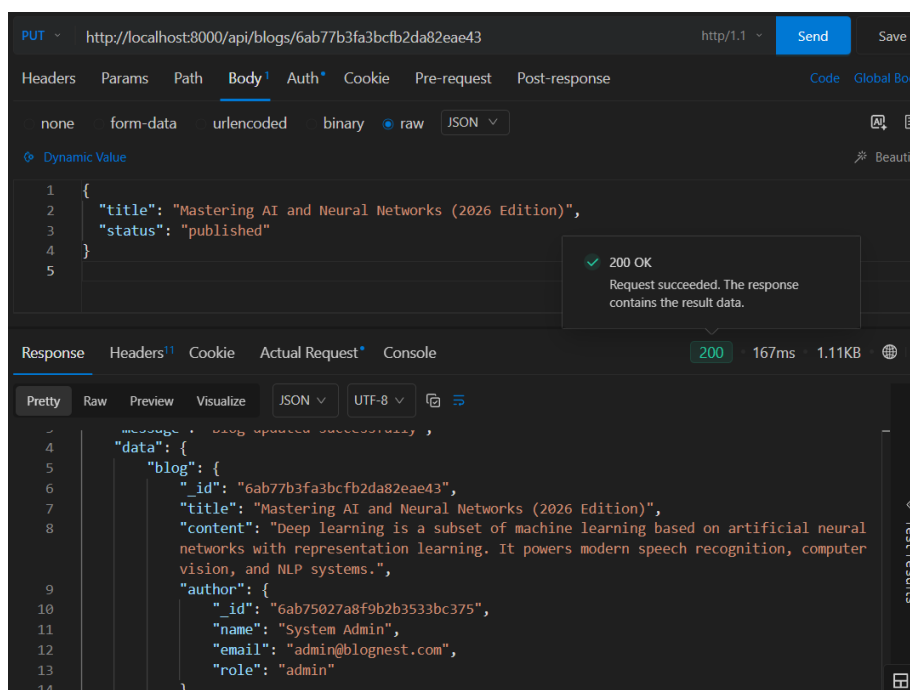


Figure 9. Blog update request returning 200 OK with the revised title and published status.

8. Category and Comment Demonstration

The category APIs demonstrate organization of blog content.

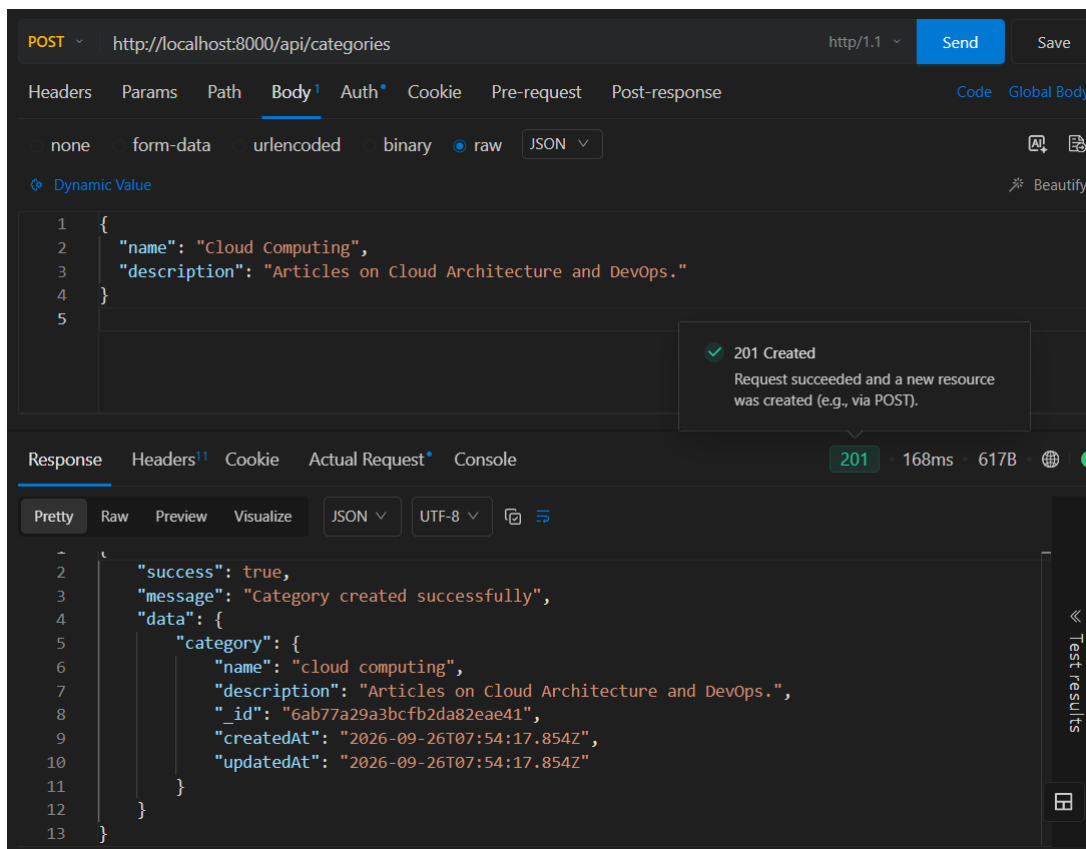


Figure 10. Category creation request returning 201 Created for the Cloud Computing category.

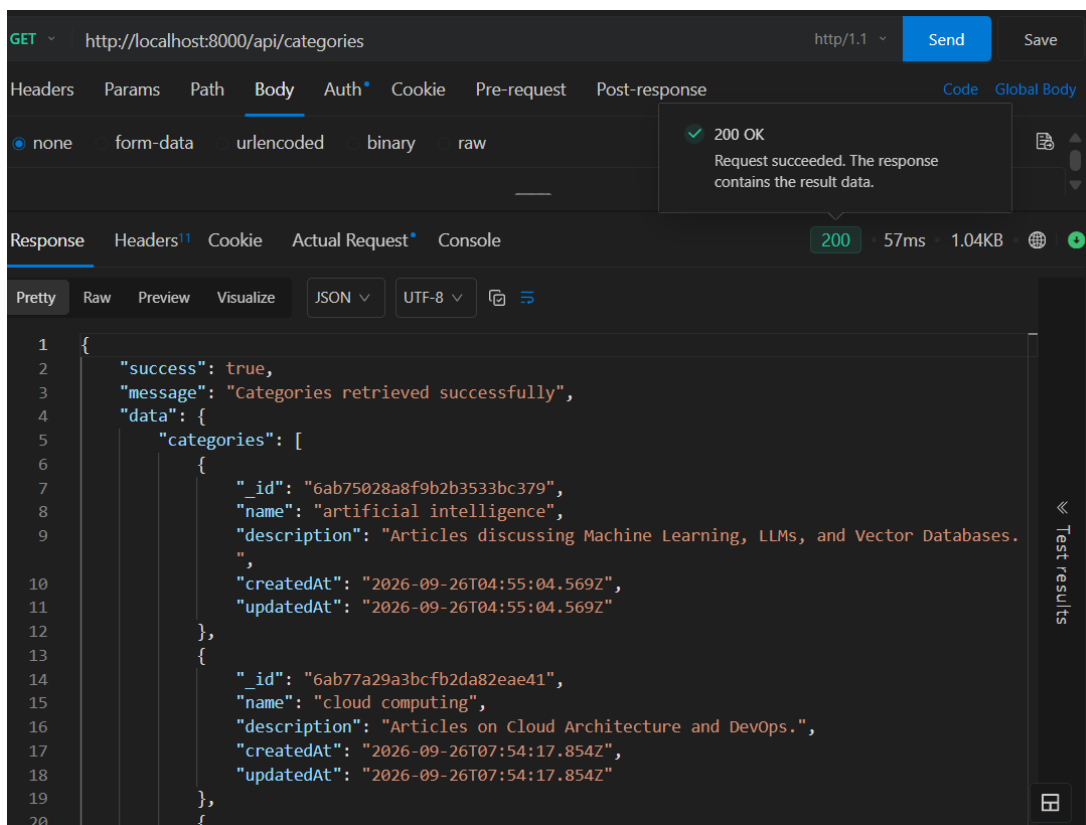


Figure 11. Category retrieval request returning 200 OK and the available categories.

8 Category and Comment Demonstration continued

The comment APIs demonstrate user interaction and moderation. Comments can be managed using moderation states such as:

- Pending
- Approved
- Spam

Editors and Administrators can perform moderation according to their permissions.

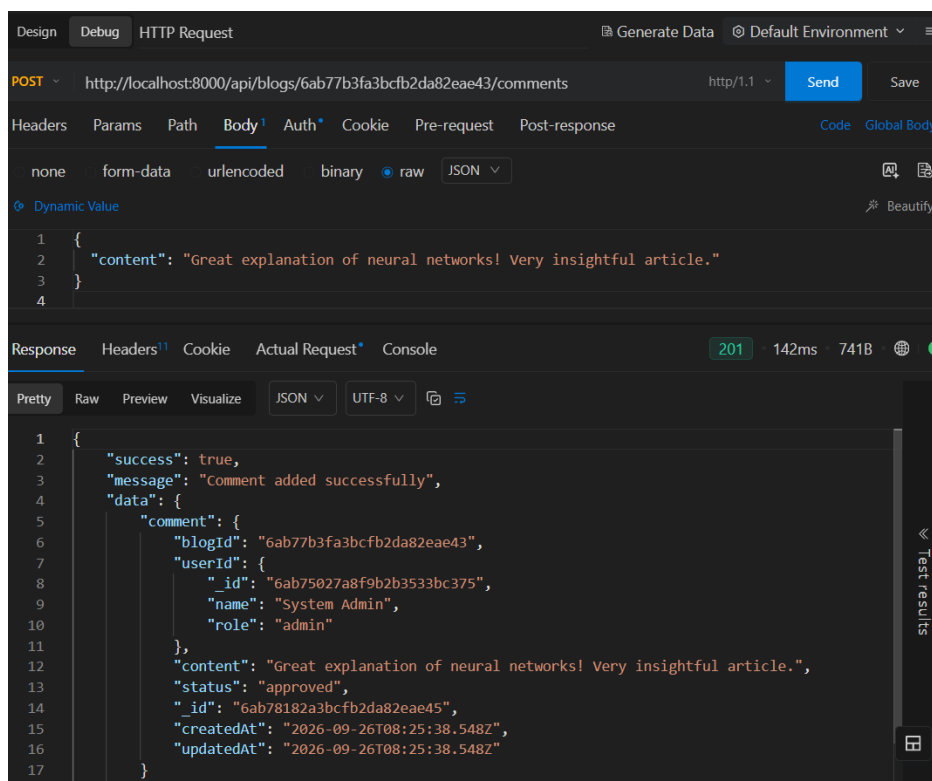


Figure 12. Comment creation request returning 201 Created with the saved comment.

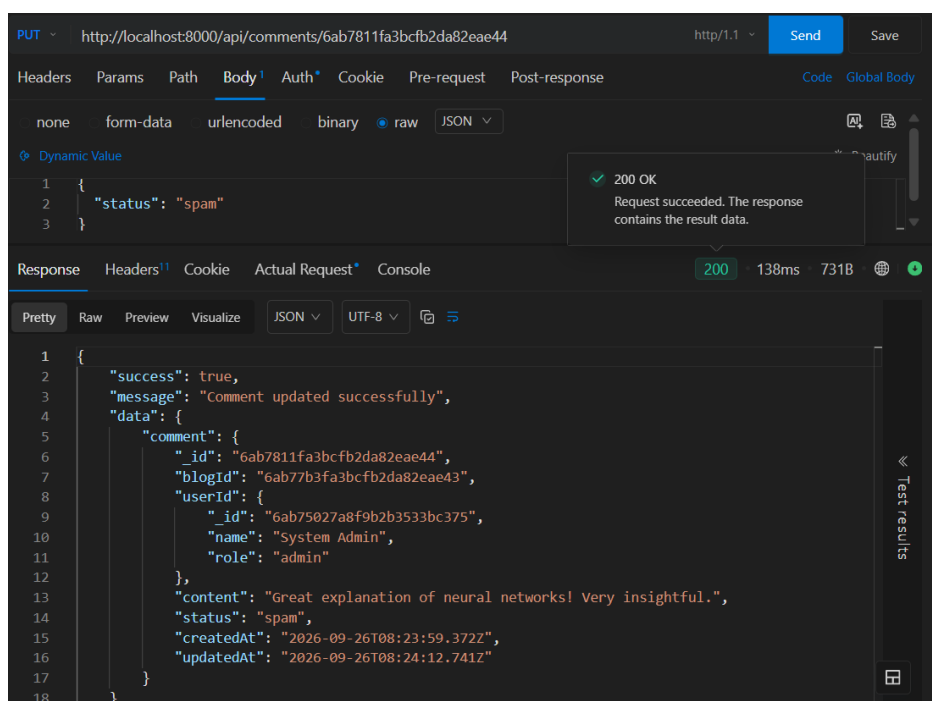


Figure 13. Comment moderation request setting the status to spam and returning 200 OK.

9. AI Blog Generation Demonstration

Google Gemini is used to generate blog content from a topic.

POST /api/ai/generate-blog

Example input:

```
{
  "topic": "Artificial Intelligence in Education"
}
```

The API returns AI-generated blog information such as title, content, summary, and tags.

The BlogNest specification specifically defines the AI blog-generation endpoint.

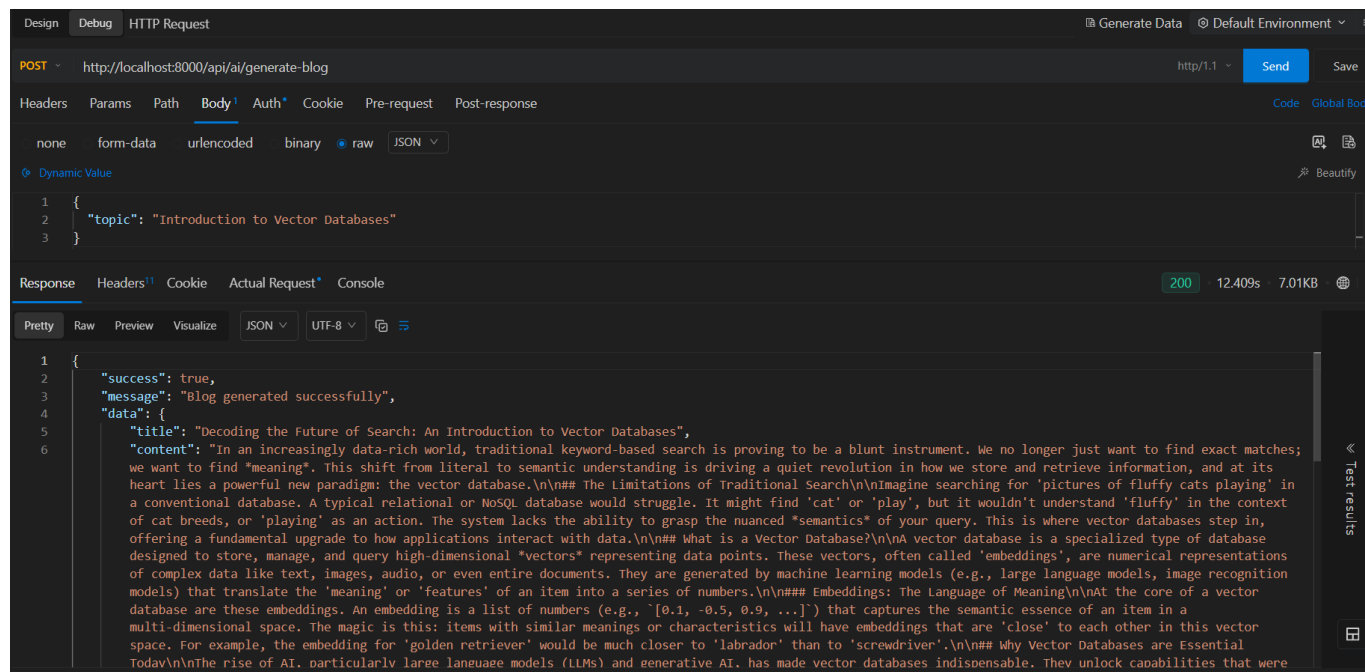


Figure 14. Gemini blog-generation request using the topic Introduction to Vector Databases and returning generated content.

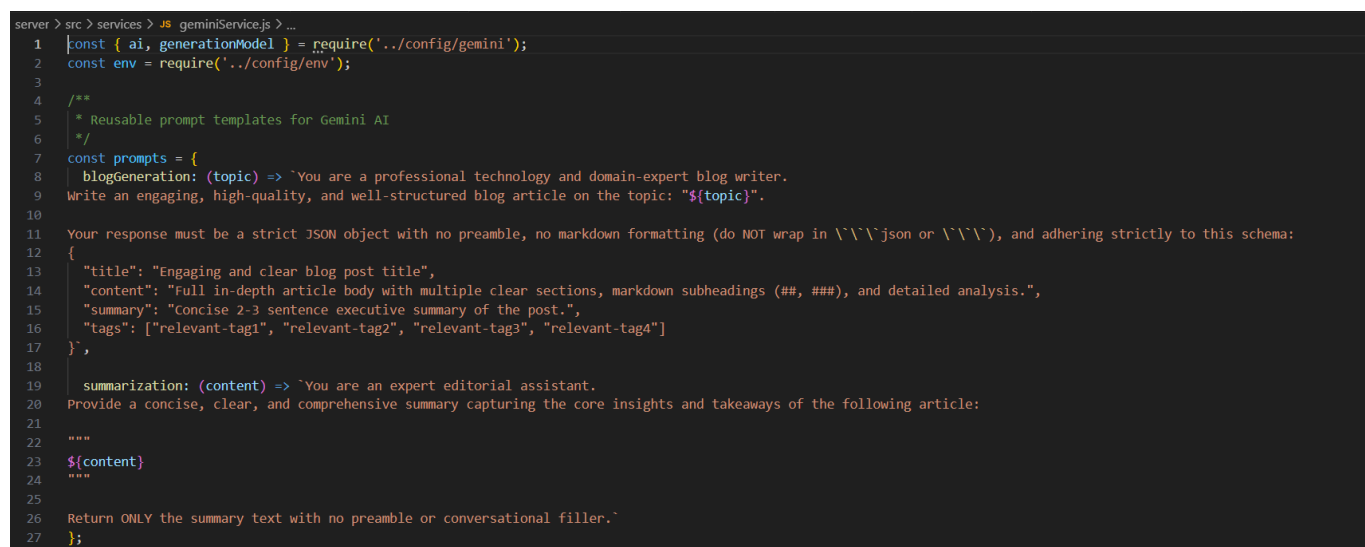


Figure 15. Prompt templates defining the generated blog structure and summarization instructions.

10. AI Summarization Demonstration

The summarization API accepts blog content and generates a concise summary.

POST /api/ai/summarize

This demonstrates the integration of Google Gemini into the backend application.

The BlogNest project demonstration specifically includes blog summarization.

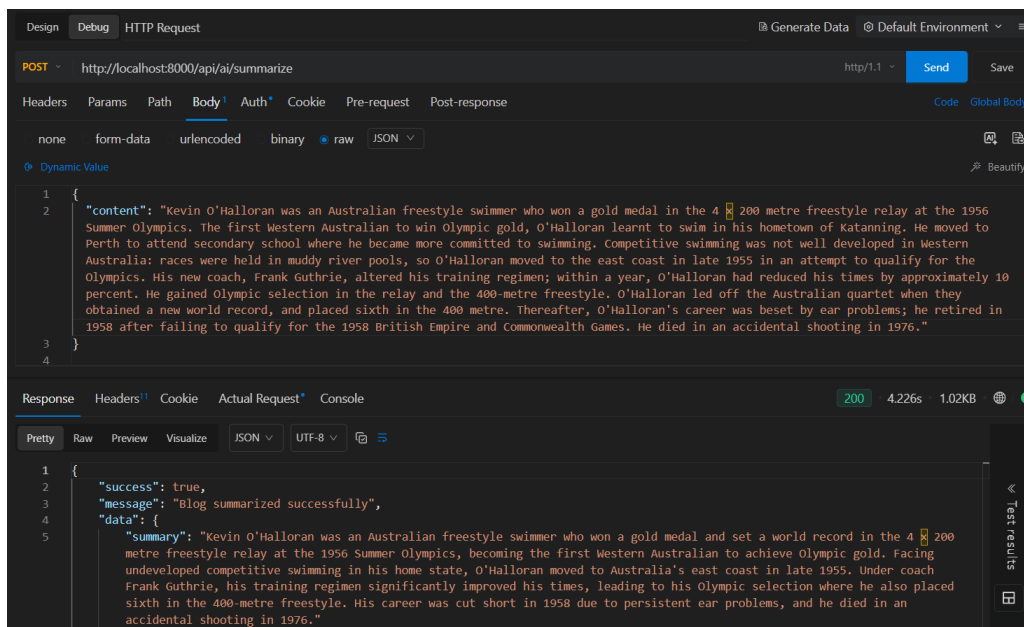


Figure 16. AI summarization request returning 200 OK and a concise summary of the supplied content.

11. Search Demonstration

The project provides two search approaches.

Keyword Search

GET /api/search?q=<search-term>

This uses MongoDB Atlas Search to find matching blog content.

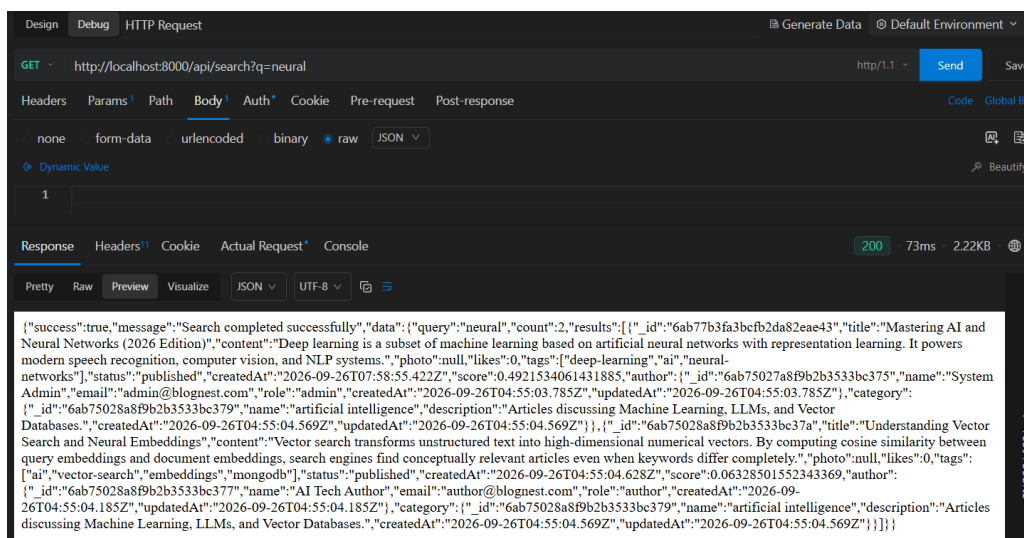


Figure 17. Keyword search request for neural returning matching blog content through MongoDB Atlas Search.

11 Search Demonstration continued

Semantic Search

GET /api/search/semantic?q=<query>

The query is converted into an embedding and processed using MongoDB Atlas Vector Search to retrieve semantically relevant blogs.

The common TN Skills requirements include both Atlas Search and Atlas Vector Search along with semantic search.

Search & Vector Search

Database	Collection	Index Name	Status	Queryable	Type	Index Fields	Documents (Estimated)	Total Size	Reco Mem
ai_blognest	blogs	default	READY View status details	✓	search	"content", "status", "tags", "title"	3 (100%) indexed of 3	14.88KB	-
ai_blognest	embeddings	vector_index	READY View status details	✓	vectorSearch	"vector"	1 (100%) indexed of 1	6.78KB	6.78KB

Figure 18. MongoDB Atlas Search and Vector Search indexes shown as ready and queryable.

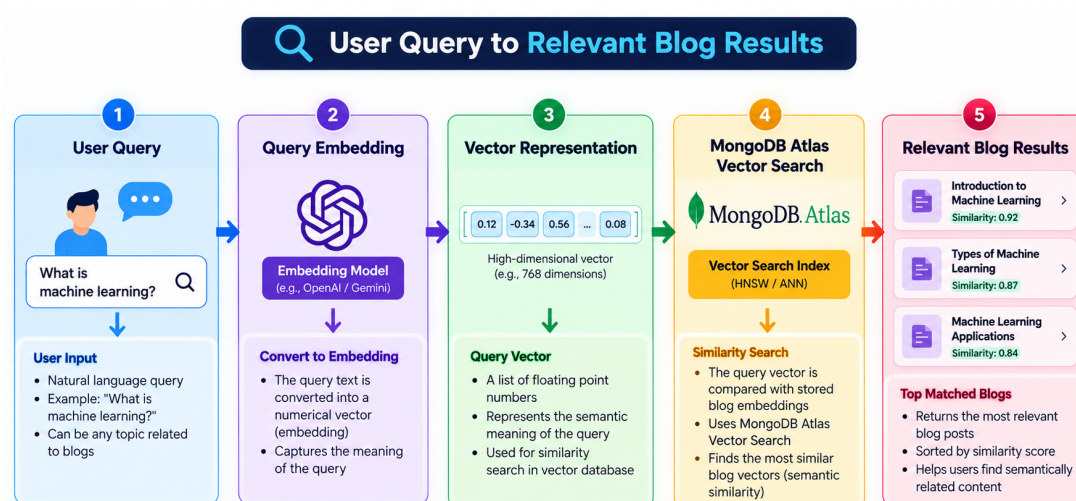


Figure 19. Semantic search flow from the user query through embedding generation and vector search to relevant blogs.

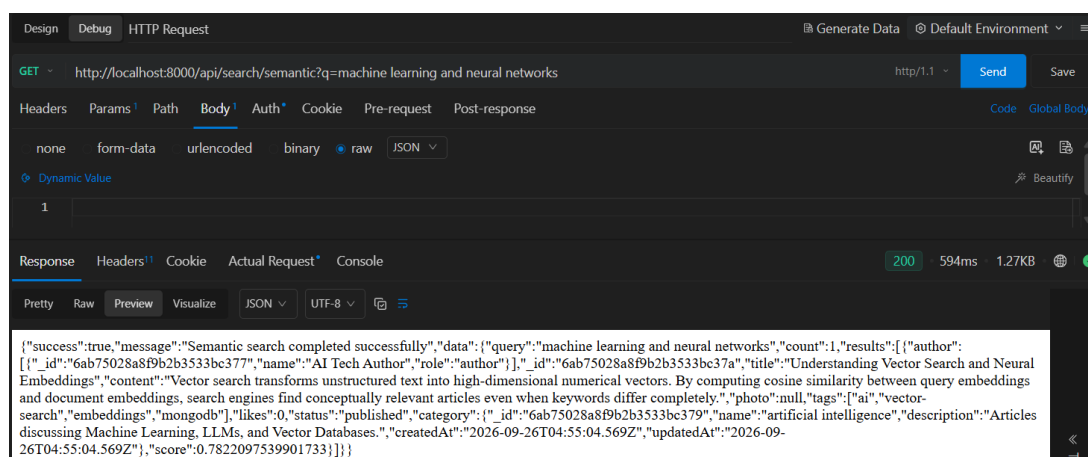


Figure 20. Semantic search request for machine learning and neural networks returning a relevant blog result.

12. Demonstration Outcome

The demonstration verifies the integrated operation of the AI BlogNest backend. The following functionality is demonstrated:

- User authentication
- JWT-based protected access
- Role-based authorization
- Blog management
- Category management
- Comment management and moderation
- AI blog generation
- AI summarization
- Keyword search
- Semantic search
- MongoDB Atlas Vector Search

The completed Phase 2 implementation was also verified through automated tests, with the reported result of 25 passed and 0 failed for that phase.

13. Conclusion

The AI BlogNest API successfully demonstrates a complete, modular backend system that brings together secure authentication, role-based access control, structured blog and content management, and AI-assisted content generation within a single cohesive application.

Across the planning, testing, and demonstration phases, the project has shown that:

- The MVC and service-based architecture cleanly separates authentication, blog management, AI services, and search functionality.
- Role-based authorization (Admin, Editor, Author, Reader) is enforced consistently across all protected endpoints.
- Google Gemini integration reliably supports both AI blog generation and summarization.
- MongoDB Atlas Search and Atlas Vector Search together provide both keyword-based and semantic content discovery.
- All 25 UAT test cases passed with zero defects, confirming the system meets its functional and non-functional requirements.

Overall, the demonstration confirms that the AI BlogNest API is a stable, working backend ready for final review and deployment. Future enhancements could include analytics on search and AI usage, richer comment-notification features, and expanded automated test coverage for edge cases.

AI BlogNest API — Project Demonstration Phase — Team SWTID-2026-6999